

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

Отчет по практической работе 6

По дисциплине Web-программирование

Тема работы Работа с сокетами

Обучающийся Ершов Николай Евгеньевич

Факультет факультет инфокоммуникационных технологий

Группа K3320

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникационных
системах

Обучающийся

(дата)

(подпись)

Ершов Н.Е.
(Ф.И.О.)

Руководитель

(дата)

(подпись)

Марченко Е.В.
(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 Задание 1	4
2 Задание 2	6
3 Задание 3	7
4 Задание 4	8
ЗАКЛЮЧЕНИЕ	9

ВВЕДЕНИЕ

Целью данной лабораторной работы является ознакомление с библиотекой socket в python.

Из составленной цели были выдвинуты следующие задачи:

- Реализовать клиентскую и серверную часть приложения;
- Реализовать решение математической задачи посредством ввода данных клиентом;
- Реализовать логику подключения клиента к серверу через браузер;
- Реализовать двухпользовательский или многопользовательский чат;

1 Задание 1

В этом задании необходимо реализовать клиентскую и серверную часть приложения, клиент должен отправлять серверу сообщение: "Hello, server!" и оно должно отобразиться на стороне сервера. В ответ на сообщение пользователю приходит ответ: "Hello, client!".

Для реализации этого задания была спроектирована архитектура проекта. Был создан класс Server, который отвечает за запуск сервера и подключение клиентов к нему. Код представлен ниже:

```
class Server:

    def __init__(self):
        self.socket_server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.socket_server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.socket_server.bind((HOST, PORT))
        self.socket_server.listen()
        self.socket_server.setblocking(False)

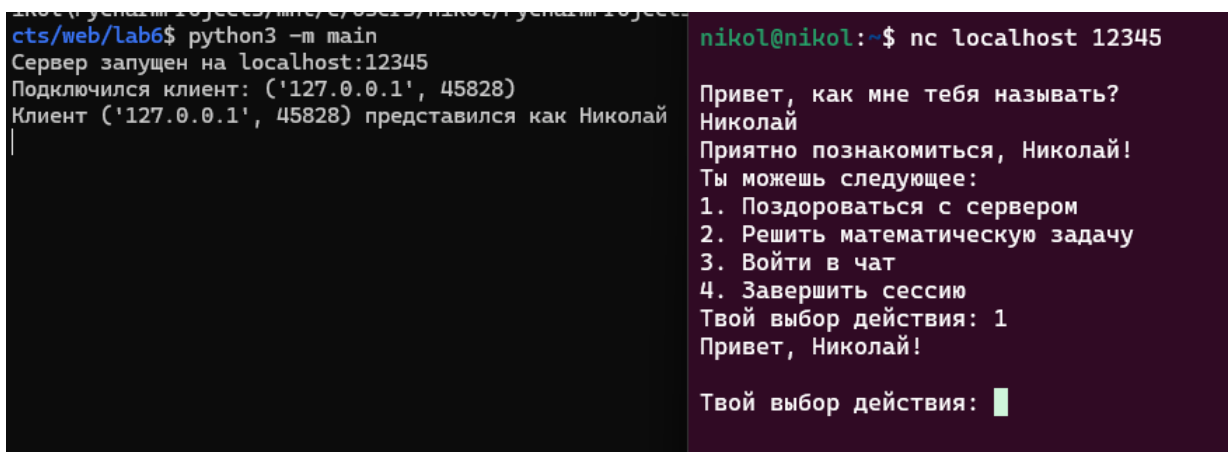
        self.handler: CommandHandler = CommandHandler()

    async def handle_client(self, client_socket, addr):
        """Обработчик клиента"""
        print(f"Подключился клиент: {addr}")
        session = ClientSession(client_socket, addr, self.handler)
        await session.process()

    async def start_server(self):
        """Запуск сервера"""
        print(f"Сервер запущен на {HOST}:{PORT}")
        loop = asyncio.get_event_loop()

        while True:
            client_socket, addr = await loop.sock_accept(self.socket_server)
            asyncio.create_task(self.handle_client(client_socket, addr))
```

У этого класса имеются два асинхронных метода: `handle_client` (отвечает за обработку подключения клиента к серверу) и `start_server` (запускает сервер). В `handle_client` создается объект класса `ClientSession` - он отвечает за обработку команд клиента. В нем есть такие методы, как: `_handle_greeting`, `_handle_math_operation`, `_handle_chat`, `_send_http_response`. В них содержится реализация исполняемых команд. В методе `process` содержится цикл обрабатывающий ответ клиента и вызывающий методы выше, исходя из пришедшего запроса клиента. Кроме того, был создан класс `CommandHandler`, который содержит логику команд (поприветствовать пользователя, решить математическую задачу и т.д.). Так, результат выполнения команды приветствия изображен на рисунке 1.1.



```
cts/web/lab6$ python3 -m main
Сервер запущен на localhost:12345
Подключился клиент: ('127.0.0.1', 45828)
Клиент ('127.0.0.1', 45828) представился как Николай

nikol@nikol:~$ nc localhost 12345
Привет, как мне тебя называть?
Николай
Приятно познакомиться, Николай!
Ты можешь следующее:
1. Поздороваться с сервером
2. Решить математическую задачу
3. Войти в чат
4. Завершить сессию
Твой выбор действия: 1
Привет, Николай!

Твой выбор действия: █
```

Рисунок 1.1 — Результат приветствия пользователя

2 Задание 2

В этом задании нужно было реализовать серверную часть приложения - решение математической операции. Пользователю на выбор дается 4 варианта задач: теорема Пифагора, решение квадратного уравнения, поиск площади трапеции, поиск площади параллелограмма.

Логика решения математических операций лежит в классе MathOperations и вызывается в CommandHandler. Пример решения квадратного уравнения (рисунок 2.1).

```
Твой выбор действия: 2
Отлично! Какую математическую задачу хочешь решить:
1. Нахождение гипотенузы треугольника по теореме Пифагора
2. Решение квадратного уравнения
3. Поиск площади трапеции
4. Поиск площади параллелограмма
Твой вариант: 2
Введите требуемые параметры через пробел.
1. a, b
2. a, b, c
3. a, b, h
4. a, h
Твои параметры: 4 4 1
Найдено одно решение:  $x = -0.5$ 
```

Рисунок 2.1 — Результат решения квадратного уравнения

3 Задание 3

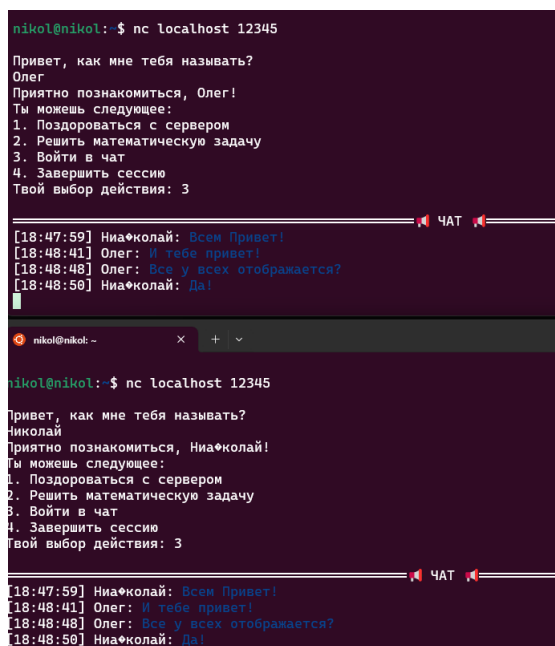
В этом задании нужно было реализовать серверную часть приложения - отправка http сообщения, содержащее html-страницу, которую сервер подгружает из index.html (рисунок 3.1).

Привет от Socket-сервера!

Рисунок 3.1 — HTML страница из index.html

4 Задание 4

В этом задании нужно было реализовать многопользовательский чат. Для этого было создано хранилище Repository, в котором хранятся сообщения пользователей и их сессии (объекты ClientSession). Также была реализована логика в CommandHandler и обработка сообщений в ClientSession. После написания кода, получился следующий результат (рисунок 4.1).



```
nikol@nikol:~$ nc localhost 12345
Привет, как мне тебя называть?
Олег
Приятно познакомиться, Олег!
Ты можешь следующее:
1. Поздороваться с сервером
2. Решить математическую задачу
3. Войти в чат
4. Завершить сессию
Твой выбор действия: 3

[18:47:59] Ниа*колай: Всем Привет!
[18:48:41] Олег: И тебе привет!
[18:48:48] Олег: Все у всех отображается?
[18:48:50] Ниа*колай: Да!

nikol@nikol:~$ nc localhost 12345
Привет, как мне тебя называть?
Николай
Приятно познакомиться, Ниа*колай!
Ты можешь следующее:
1. Поздороваться с сервером
2. Решить математическую задачу
3. Войти в чат
4. Завершить сессию
Твой выбор действия: 3

[18:47:59] Ниа*колай: Всем Привет!
[18:48:41] Олег: И тебе привет!
[18:48:48] Олег: Все у всех отображается?
[18:48:50] Ниа*колай: Да!
```

Рисунок 4.1 — Многопользовательский чат

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы были выполнены следующие задачи:

- Реализованы клиентская и серверная часть приложения;
- Реализовано решение математической задачки посредством ввода данных клиентом;
- Реализована логика подключения клиента к серверу через браузер;
- Реализованы двухпользовательский или многопользовательский чат;

А значит, цель, озвученная в начале - ознакомление с библиотекой `socket` в `python` была достигнута.